

UNIVERSITÀ DEGLI STUDI DI
UDINE

DIPARTIMENTO DI SCIENZE
MATEMATICHE, INFORMATICHE E FISICHE

Anno accademico 2024/2025



Algoritmi e Strutture dati Progetto di Laboratorio

Autori

Trentin Tommaso, 167340

Cuna Davide, 168363

trentin.tommaso@spes.uniud.it

cuna.davide@spes.uniud.it

Contents

1	Introduzione	3
1.1	Obiettivi del progetto	3
1.1.1	Implementazione degli algoritmi	3
1.1.2	Misurazione dei tempi medi di esecuzione al variare di n e m	3
1.2	Algoritmi Analizzati	4
1.2.1	QuickSort	4
1.2.2	QuickSort3way	5
1.2.3	CountingSort	6
1.2.4	MergeSort	7
2	Esperimenti e Risultati	8
2.1	Setup degli esperimenti	8
2.2	Grafici in funzione di n (con m costante)	10
2.2.1	Grafici comparativi	10
2.3	Grafici in funzione di m (con n costante)	11
2.3.1	Grafici Comparativi	11
2.4	Osservazioni qualitative	12
3	Conclusioni	13
3.1	Algoritmo più efficiente per range diversi	13
3.2	Tabella riassuntiva degli algoritmi	13

1 Introduzione

1.1 Obiettivi del progetto

L'obiettivo di questo progetto di laboratorio consiste nell'analisi sperimentale delle prestazioni di quattro tipi diversi di algoritmi attraverso l'implementazione dello pseudocodice visto a lezione in un linguaggio da noi scelto (in questo caso C) e la misurazione dei tempi medi di esecuzione al variare dei due parametri 'n' e 'm', rispettivamente la lunghezza dell'array in input da ordinare e l'intervallo degli interi che apparivano nel vettore in questione. L'attività svolta si articola in due fasi principali:

1.1.1 Implementazione degli algoritmi

Le implementazioni dei seguenti algoritmi sono state sviluppate in C e successivamente testate su appositi moduli VPL:

- **QuickSort**, basato su partizionamento ricorsivo
- **QuickSort 3-way**, versione ottimizzata del precedente per input con molti duplicati
- **CountingSort**, algoritmo non comparativo e lineare
- **MergeSort**, algoritmo stabile "divide et impera"

1.1.2 Misurazione dei tempi medi di esecuzione al variare di n e m

Questa seconda parte del progetto prevede la misurazione dei tempi medi di esecuzione di ciascun algoritmo su array casuali e generati in maniera controllata. Le dimensioni dell'array n variano nel range $100 \rightarrow 100000$, mentre il range degli interi m è compreso tra $10 \rightarrow 1000000$. Entrambi i parametri n e m sono stati campionati tramite progressione (serie) geometrica, generando 1000 punti per ciascun esperimento.

Per ogni combinazione (n, m) , si generano array casuali con valori uniformemente distribuiti nel range $[1, \dots, m]$ e l'algoritmo di ordinamento è stato testato ripetutamente fino al superamento di un tempo minimo misurabile T_{min} , stimato in base alla risoluzione del clock di sistema. Il tempo medio per ciascun input è stato ottenuto successivamente come rapporto

$$\frac{\text{tempo totale delle iterazioni}}{\text{numero di iterazioni}}$$

I dati raccolti sono stati successivamente usati per la produzione di grafici comparativi e l'analisi del comportamento asintotico degli algoritmi in funzione dei parametri n e m .

1.2 Algoritmi Analizzati

1.2.1 QuickSort

```
1  #include <stdio.h>
2
3  // Funzione per scambiare due elementi nell'array.
4  void Swap(int *A, int i, int j) {
5      int tmp = A[i];
6      A[i] = A[j];
7      A[j] = tmp;
8  }
9
10 // Funzione per partizionare l'array nella versione classica di QuickSort.
11 int Partition(int *A, int p, int q) {
12     int x = A[q]; // Sceglie l'ultimo elemento come pivot.
13     int i = p - 1; // Indice per gli elementi minori del pivot.
14
15     // Partiziona l'array in base al pivot.
16     for (int j = p; j < q; j++) {
17         if (A[j] < x) { // Elemento minore del pivot.
18             i++;
19             Swap(A, i, j); // Sposta l'elemento nella parte sinistra.
20         }
21     }
22     i++;
23     Swap(A, i, q); // Posiziona il pivot nella posizione corretta.
24     return i; // Restituisce l'indice del pivot.
25 }
26
27 // Implementazione di QuickSort con Partition.
28 void QuickSort(int *A, int low, int high) {
29     if (low < high) { // Continua solo se ci sono almeno due elementi.
30         int r = Partition(A, low, high); // Partiziona l'array e ottiene il pivot.
31         QuickSort(A, low, r - 1); // Ordina la parte sinistra.
32         QuickSort(A, r + 1, high); // Ordina la parte destra.
33     }
34 }
```

1.2.2 QuickSort3way

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Funzione per scambiare due elementi nell'array.
5  void Swap3w(int *A, int i, int j) {
6      int tmp = A[i];
7      A[i] = A[j];
8      A[j] = tmp;
9  }
10
11 /*
12  * Implementazione di QuickSort a tre vie (3-way QuickSort).
13  * Questo approccio particolarmente utile per array con molti elementi ripetuti.
14  * La partizione a tre vie divide l'array in tre sezioni:
15  * - Elementi minori del pivot.
16  * - Elementi uguali al pivot.
17  * - Elementi maggiori del pivot.
18  * Questo riduce il numero di confronti e scambi per gli elementi uguali al pivot,
19  * migliorando le prestazioni in caso di molti duplicati.
20  */
21 void QuickSort3w(int *A, int low, int high) {
22     if (low >= high) return; // Caso base: array con uno o zero elementi.
23
24     int pivot = A[low]; // Sceglie il primo elemento come pivot.
25     int lt = low;       // Indice per la sezione degli elementi minori del pivot.
26     int gt = high;      // Indice per la sezione degli elementi maggiori del pivot.
27     int i = low + 1;    // Indice per scansionare l'array.
28
29     // Partizione a tre vie.
30     while (i <= gt) {
31         if (A[i] < pivot) { // Elemento minore del pivot.
32             Swap3w(A, lt, i); // Sposta l'elemento nella sezione dei minori.
33             lt++;
34             i++;
35         } else if (A[i] > pivot) { // Elemento maggiore del pivot.
36             Swap3w(A, i, gt); // Sposta l'elemento nella sezione dei maggiori.
37             gt--;
38         } else { // Elemento uguale al pivot.
39             i++;
40         }
41     }
42
43     // Ordina ricorsivamente le sezioni sinistra e destra.
44     QuickSort3w(A, low, lt - 1); // Ordina la parte sinistra (minori del pivot).
45     QuickSort3w(A, gt + 1, high); // Ordina la parte destra (maggiori del pivot).
46 }
```

1.2.3 CountingSort

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void CountingSort(int *A, int n) {
5      // Caso base: se l'array vuoto o di dimensione non valida, termina la funzione.
6      if (n <= 0) return;
7
8      // Trova il valore massimo (M) e minimo (m) nell'array.
9      int M = A[0], m = A[0];
10     for (int i = 1; i < n; i++) {
11         if (A[i] < m) m = A[i]; // Aggiorna il minimo.
12         else if (A[i] > M) M = A[i]; // Aggiorna il massimo.
13     }
14
15     // Calcola l'intervallo dei valori (range) e alloca un array di conteggio.
16     int range = M - m + 1;
17     int *countA = (int*)calloc(range, sizeof(int)); // Inizializza l'array a 0.
18     if (!countA) exit(1); // Termina il programma se l'allocazione fallisce.
19
20     // Conta la frequenza di ciascun elemento nell'array originale.
21     for (int i = 0; i < n; i++) {
22         countA[A[i] - m]++; // Usa l'indice normalizzato (A[i] - m).
23     }
24
25     // Calcola i prefissi cumulativi per determinare le posizioni finali.
26     for (int i = 1; i < range; i++) {
27         countA[i] += countA[i - 1]; // Somma cumulativa.
28     }
29
30     // Alloca un array temporaneo per l'output ordinato.
31     int *outputA = (int*)malloc(n * sizeof(int));
32     if (!outputA) exit(1); // Termina il programma se l'allocazione fallisce.
33
34     // Costruisce l'array ordinato iterando dall'ultimo elemento (stabile).
35     for (int i = n - 1; i >= 0; i--) {
36         outputA[countA[A[i] - m] - 1] = A[i]; // Posiziona l'elemento nella posizione corretta.
37         countA[A[i] - m]--; // Decrementa il conteggio per gestire duplicati.
38     }
39
40     // Copia l'array ordinato nell'array originale.
41     for (int i = 0; i < n; i++) {
42         A[i] = outputA[i];
43     }
44
45     // Libera la memoria allocata dinamicamente.
46     free(countA);
47     free(outputA);
48 }
```

1.2.4 MergeSort

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void merge(int *A, int low, int middle, int high) {
5      int i, j, k;
6      int n1 = middle - low + 1; // Dimensione del sottoarray sinistro.
7      int n2 = high - middle;    // Dimensione del sottoarray destro.
8
9      // Alloca memoria per i due sottoarray (L = lower e H = higher).
10     int *L = (int *)malloc(n1 * sizeof(int));
11     int *H = (int *)malloc(n2 * sizeof(int));
12
13     for (i = 0; i < n1; i++) {    // Copia i dati nel sottoarray sinistro (L).
14         L[i] = A[low + i];
15     }
16     for (j = 0; j < n2; j++) {    // Copia i dati nel sottoarray destro (H).
17         H[j] = A[middle + 1 + j];
18     }
19
20     i = 0; // Indice per il sottoarray sinistro.
21     j = 0; // Indice per il sottoarray destro.
22     k = low; // Indice per l'array principale.
23
24     while (i < n1 && j < n2) {    // Confronta e unisce i due sottoarray in ordine crescente.
25         if (L[i] <= H[j]) { // Se l'elemento in L minore o uguale.
26             A[k] = L[i];
27             i++;
28         } else { // Altrimenti, prendi l'elemento in H.
29             A[k] = H[j];
30             j++;
31         }
32         k++;
33     }
34
35     while (i < n1) {    // Copia gli elementi rimanenti del sottoarray sinistro (se presenti).
36         A[k] = L[i];
37         i++;
38         k++;
39     }
40
41     while (j < n2) {    // Copia gli elementi rimanenti del sottoarray destro (se presenti).
42         A[k] = H[j];
43         j++;
44         k++;
45     }
46
47     free(L);    // Libera la memoria allocata per sottoarray L.
48     free(H);    // Libera la memoria allocata per sottoarray.
49 }
50
51 void MergeSort(int *A, int low, int high) {
52     if (low < high) { // Condizione base: continua solo se ci sono almeno due elementi.
53         int middle = low + (high - low) / 2; // Calcola il punto medio per dividere l'array.
54
55         MergeSort(A, low, middle);    // Ordina ricorsivamente la met sinistra.
56         MergeSort(A, middle + 1, high); // Ordina ricorsivamente la met destra.
57         merge(A, low, middle, high);  // Unisce le due met ordinate.
58     }
59 }
60 }
```

2 Esperimenti e Risultati

2.1 Setup degli esperimenti

Per ogni test effettuato verrà generato in modo pseudo-casuale (mediante la funzione *rand()*) un array con n lunghezza e m range dati.

```
1 void fill_array(int *A, int size, int range) {
2     for (int i = 0; i < size; i++)
3         A[i] = rand() % range + 1;
4 }
```

Per le misurazioni dei tempi di esecuzione si utilizza la funzione *QueryPerformanceFrequency* (importata tramite la libreria windows.h) in quanto, successivamente a una prima prova con la funzione *time()*, si è notato come quest'ultima avesse una risoluzione troppo lunga rispetto a quella degli algoritmi in caso di n molto piccoli. *QueryPerformanceFrequency* garantisce invece maggiore precisione e ha permesso di effettuare misurazioni più accurate.

```
1 double get_time() {
2     static LARGE_INTEGER freq = {0};
3     LARGE_INTEGER now;
4     if (freq.QuadPart == 0) {
5         QueryPerformanceFrequency(&freq);
6     }
7     QueryPerformanceCounter(&now);
8     return (double)now.QuadPart / freq.QuadPart;
9 }
```

Nella funzione *test_n()*, per ogni test da eseguire, si alloca inizialmente la memoria necessaria agli array per ciascun algoritmo e successivamente si vanno a riempire mediante la funzione *fill_array()*. In seguito, per ciascun test si eseguono *REP* ripetizioni (25) di ciascun algoritmo sugli array generati e se ne segna il tempo di esecuzione. Alla fine delle ripetizioni si effettua una media dei tempi che viene segnata nei file .csv di output (lo scopo di queste ripetizioni è minimizzare eventuali errori di misurazione che si tradurrebbero in "salti" nei grafici esposti in seguito). La dimensione degli array n varia da 100 a 100000 e che, con il susseguirsi dei test, cresce seguendo una serie geometrica in questo intervallo di valori. La dimensione del range di valori m è invece fissata a 100000.

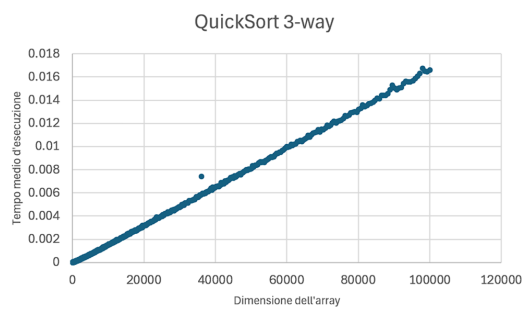
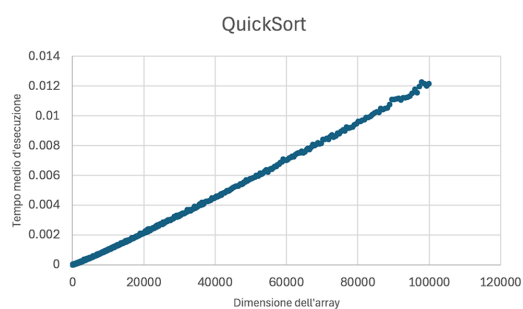
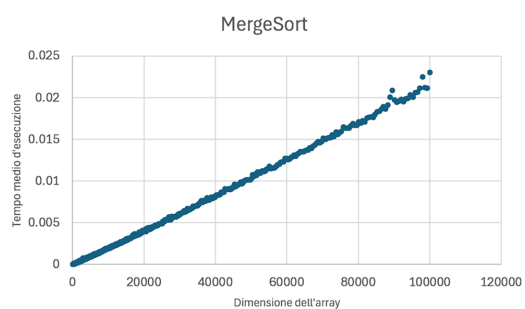
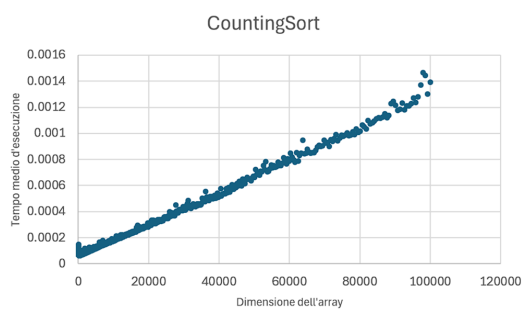
Si ha poi una funzione *test_m()* che effettua invece test al variare di m . La logica del test è conforme a quella di *test_n()*, si avrà però un valore di n fissato a 10000 ed un valore di m che invece crescerà, sempre secondo una serie geometrica tra 10 e 1000000.

```

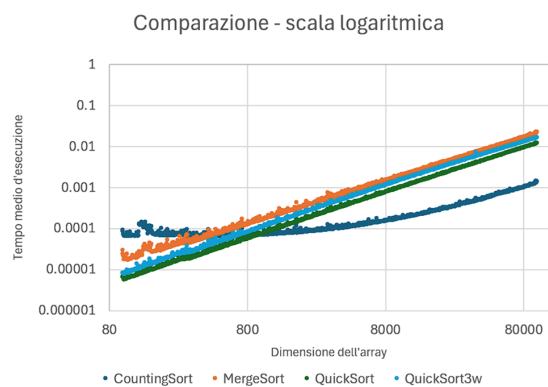
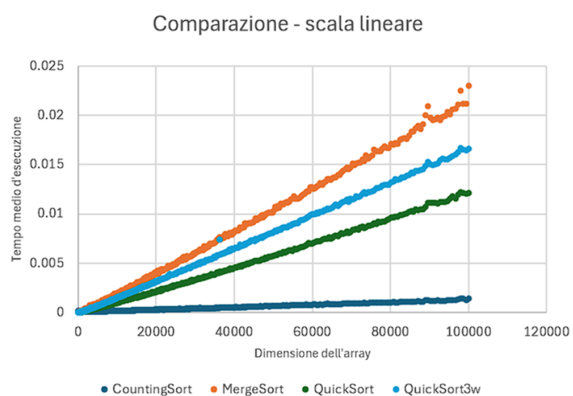
1 void test_n() {
2     FILE *f = fopen("benchmark_n.csv", "w");
3     if (!f) {
4         printf("Error opening file for writing\n");
5         exit(1);
6     }
7     fprintf(f, "n,CountingSort,MergeSort,QuickSort,QuickSort3w\n");
8
9     srand(time(NULL));
10    double A = N_MIN;
11    double B = pow((double)N_MAX / N_MIN, 1.0 / (SAMPLES - 1));
12    double t_counting, t_merge, t_quick, t_quick3w;
13
14    for (int i = 0; i < SAMPLES; i++) {
15        int n = floor(A * pow(B, i));
16
17        t_counting = 0.0;
18        t_merge = 0.0;
19        t_quick = 0.0;
20        t_quick3w = 0.0;
21
22        int *A1 = malloc(n * sizeof(int));
23        int *A2 = malloc(n * sizeof(int));
24        int *A3 = malloc(n * sizeof(int));
25        int *A4 = malloc(n * sizeof(int));
26
27        for (int rep = 0; rep < REP; rep++) {
28            fill_array(A1, n, M_FIXED);
29            memcpy(A2, A1, n * sizeof(int));
30            memcpy(A3, A1, n * sizeof(int));
31            memcpy(A4, A1, n * sizeof(int));
32
33            if (!A1 || !A2 || !A3 || !A4) {
34                printf("malloc failed for n=%d\n", n);
35                exit(1);
36            }
37
38            double t_tmp = get_time();
39            CountingSort(A1, n);
40            t_counting += get_time() - t_tmp;
41
42            t_tmp = get_time();
43            MergeSort(A2, 0, n - 1);
44            t_merge += get_time() - t_tmp;
45
46            t_tmp = get_time();
47            QuickSort(A3, 0, n - 1);
48            t_quick += get_time() - t_tmp;
49
50            t_tmp = get_time();
51            QuickSort3w(A4, 0, n - 1);
52            t_quick3w += get_time() - t_tmp;
53        }
54
55        free(A1);
56        free(A2);
57        free(A3);
58        free(A4);
59
60        fprintf(f, "%d,%.8f,%.8f,%.8f,%.8f\n", n, t_counting / REP,
61            t_merge / REP, t_quick / REP, t_quick3w / REP);
62    }
63    fclose(f);
64    printf("Benchmarking completed. Results saved to benchmark_n.csv\n");
65 }

```

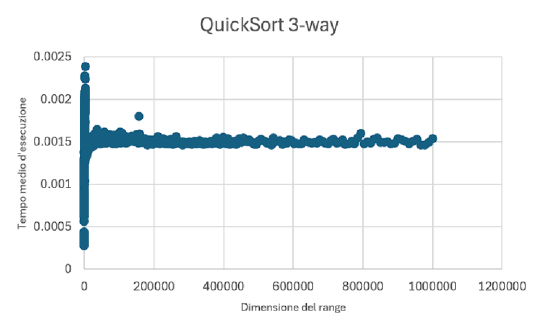
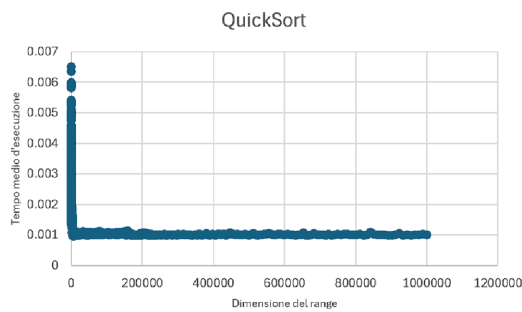
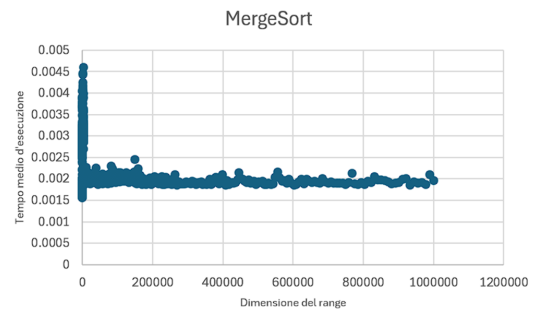
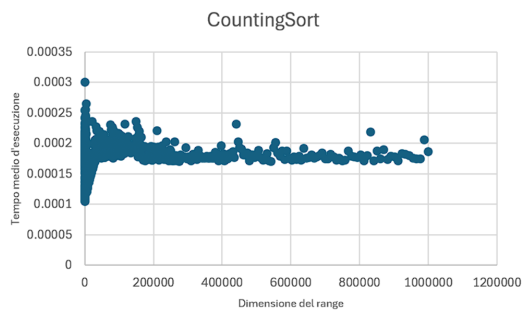
2.2 Grafici in funzione di n (con m costante)



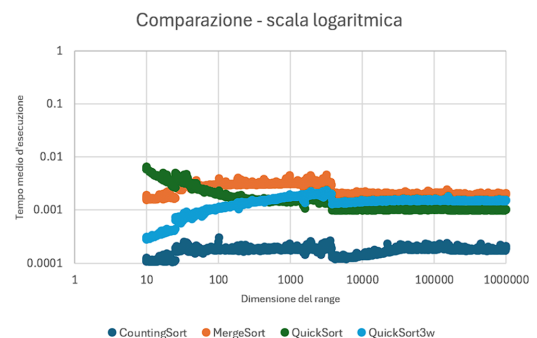
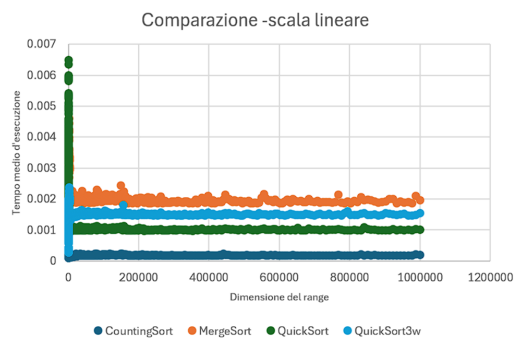
2.2.1 Grafici comparativi



2.3 Grafici in funzione di m (con n costante)



2.3.1 Grafici Comparativi



2.4 Osservazioni qualitative

L'analisi qualitativa dei risultati e dei grafici proposti ci ha permesso di evidenziare alcune caratteristiche dei diversi algoritmi di ordinamento da noi testati.

- **QuickSort**: ha mostrato prestazioni molto buone nella maggior parte dei casi mantenendo la complessità temporale attesa di $\mathcal{O}(n \log n)$. Tuttavia, il suo tempo di esecuzione varia sensibilmente in funzione della distribuzione dei dati in input. Un esempio dimostrato dai grafici precedentemente mostrati è la presenza di tante ripetizioni dovute da un range m ridotto.
- **QuickSort 3-way**: questa implementazione conferma la sua efficacia nella gestione di array con valori ripetuti, gestendo le ripetizioni utilizzando meno confronti e meno scambi rispetto alla versione standard sopra citata. Il tempo medio di esecuzione è dunque ridotto ma in assenza di numeri duplicati, la differenza tra i due algoritmi inizia a venir meno.
- **CountingSort**: come visto in teoria questo algoritmo viene fortemente influenzato dal range m . Con valori $m \gg n$ CountingSort mostra un aumento sostanziale nel tempo di esecuzione dato dalla necessità di allocare ed inizializzare strutture dati proporzionali all'intervallo dei valori ma in scenari con m ridotto, l'algoritmo in questione ha evidenziato prestazioni superiori agli algoritmi comparativi dimostrando la sua complessità lineare $\mathcal{O}(n + m)$.
- **MergeSort**: ha mostrato prestazioni stabili e costanti in tutte le configurazioni proposte confermando la sua complessità temporale di $\mathcal{O}(n \log n)$ sia nel caso peggiore che in quello migliore. L'algoritmo in questione non viene influenzato dalla presenza di duplicati e nemmeno dall'input iniziale, tuttavia richiede spazio ausiliario proporzionale a 'n' per la gestione dei vettori temporanei usati nella fase di merge.

3 Conclusioni

3.1 Algoritmo più efficiente per range diversi

L'analisi sperimentale ha evidenziato il variare dell'efficienza in funzione dei parametri n ed m . In particolare:

- **QuickSort**: ha fornito prestazioni elevate nella maggior parte dei casi testati. Tuttavia ha mostrato una certa sensibilità alla distribuzione dell'array di input con un rischio di peggioramento in situazioni sfavorevoli come array già ordinati (o quasi) per via del pivot fisso.
- **QuickSort 3-way**: ha offerto buone prestazioni anche in presenza di numerosi valori ripetuti riducendo il numero di confronti rispetto alla versione classica. In assenza di duplicati il comportamento è lo stesso del QuickSort base.
- **CountingSort**: si è dimostrato il più veloce per $m \ll n$ per via della sua complessità lineare. Tuttavia, al crescere di m il costo di inizializzazione dell'array di counting rende questo algoritmo meno conveniente rispetto agli algoritmi comparativi, specialmente per valori di m molto elevati.
- **MergeSort**: si è confermato stabile e con tempi di esecuzione crescenti in funzione di n senza però risentire dell'aumentare di m . Tuttavia, il costo aggiuntivo in termini di spazio ausiliario è una limitazione rispetto ad algoritmi in-place come QuickSort, essendo necessaria memoria proporzionale a n per la fase di merge.

3.2 Tabella riassuntiva degli algoritmi

Algoritmo	Tempo medio	Tempo peggiore	In-place	Stabile
QuickSort	$O(n \log n)$	$O(n^2)$	Sì	No
QuickSort 3-way	$O(n \log n)$	$O(n^2)$	Sì	No
CountingSort	$O(n + m)$	$O(n + m)$	No	Sì
MergeSort	$O(n \log n)$	$O(n \log n)$	No	Sì